



The perfect gift for readers and writers. [Give the gift of Medium](#)

★ Member-only story

GraphMind: A Full-Stack Agentic Architecture for Deep Retrieval and Multi-Hop Reasoning



Akash Goyal

Follow

8 min read · Nov 23, 2025

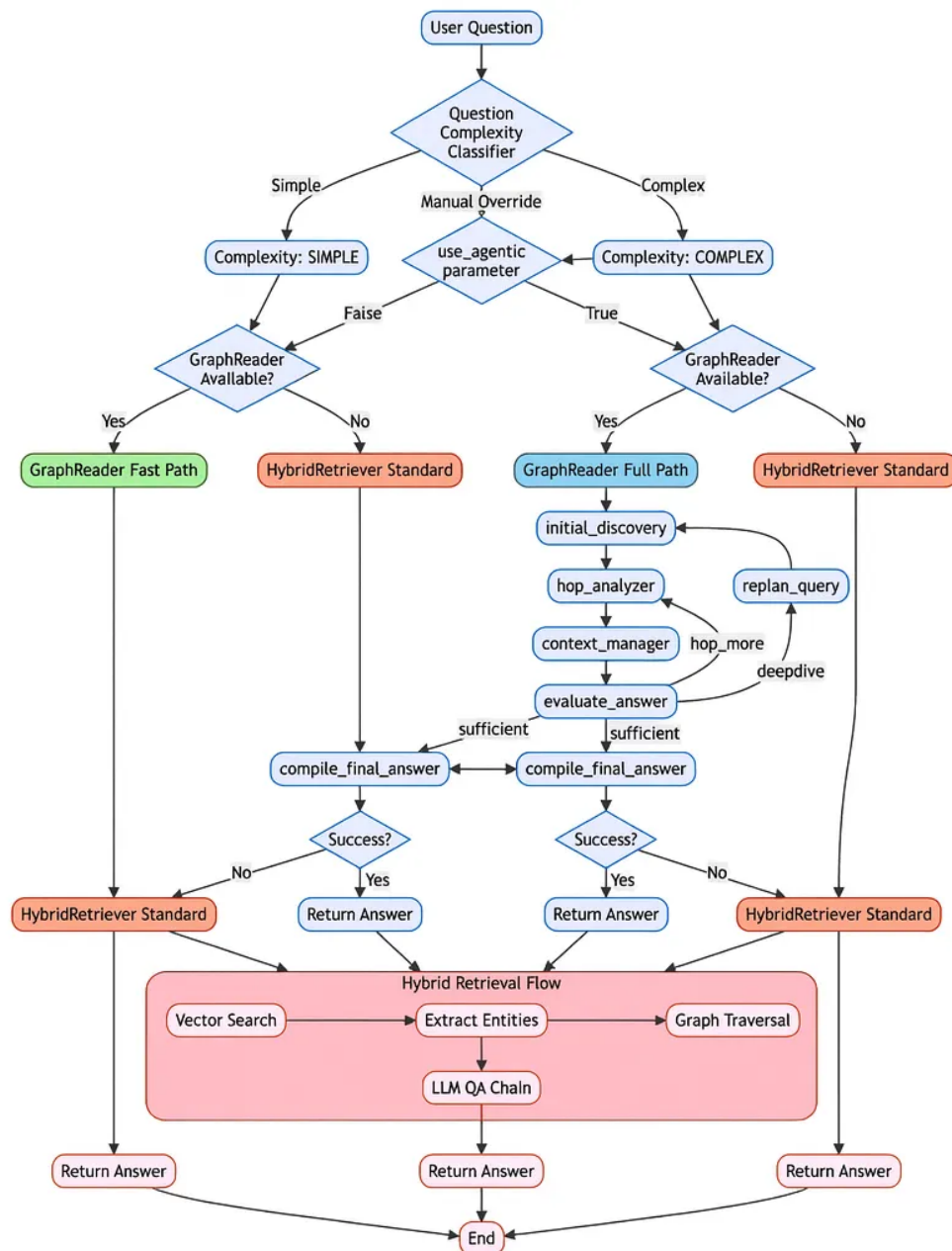


38



2

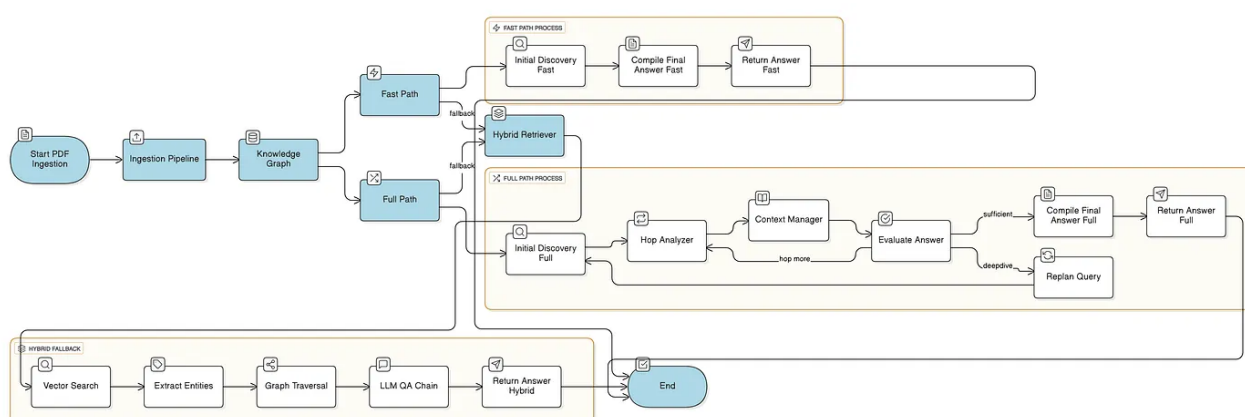




This blog walks readers through a complete, production-grade architecture for transforming complex unstructured data into highly accurate, query-ready knowledge using a GraphRAG pipeline. It demonstrates how **Chonkie** performs semantic chunking to preserve context, how **Neo4j** stores both vector and graph representations for dual-retrieval capabilities, and how **LiteLLM** orchestrates the reasoning process. The article then explains how the system uses

Intelligent Routing to dynamically switch between fast, direct answers and deep, multi-hop reasoning agents. By the end of the blog, readers will understand how to build a robust, agentic RAG system that seamlessly blends vector search, graph traversal, and orchestrated LLM reasoning.

The Architecture



The architecture begins by ingesting raw documents and processing them through **Chonkie**, a semantic chunking engine that respects sentence boundaries and logical flow, ensuring that no context is lost in the split. These chunks are then processed by **LiteLLM** to extract atomic facts and entities, which are stored in **Neo4j**. This creates a “Knowledge Graph” where data points are not just isolated vectors but interconnected nodes.

When a user submits a query, an **Intelligent Router** analyzes its complexity. Simple questions take the “Fast Path” — a direct vector

search that returns an answer in milliseconds. Complex questions trigger the “GraphReader Agent,” which performs multi-hop reasoning: traversing the graph, collecting related facts, and synthesizing a deep answer. The result is a system that is both efficient (low cost for simple queries) and highly reliable (deep reasoning for complex ones).

Why should you consider GraphRAG?

GraphRAG offers several advantages aligned with enterprise-grade infrastructure.

1. **Deep Reasoning:** Standard RAG retrieves documents based on keyword similarity. GraphRAG understands *relationships*. It knows that “PD-1” is related to “Immunotherapy” even if the documents don’t explicitly say so in the same sentence.
2. **Cost Efficiency:** By using Intelligent Routing, we don’t burn expensive tokens on simple questions. We reserve the heavy-duty agentic reasoning only for the queries that actually need it.
3. **Traceability:** Every answer is constructed from a specific path of facts. You can trace exactly which nodes the agent visited to arrive at its conclusion, reducing hallucinations.
4. **Robustness:** The system includes a “Hybrid Fallback” mechanism. If the fancy agent fails, it gracefully degrades to a standard vector search, ensuring the user always gets an answer

The Tech Stack

We use a modern, modular stack to build this “Cognitive Graph”. Each component was chosen for its specific strength in the pipeline.

- **Chonkie:** A specialized chunking library that breaks long documents into meaningful “chunks” (not just random splits), preserving semantic context.
- **PyMuPDF4LLM:** For robust PDF parsing that preserves layout and tables.
- **Neo4j:** The brain. It stores both the text (Vector) and the connections (Graph) in a single database, allowing for hybrid retrieval.
- **LiteLLM:** The reasoning engine. It provides a unified interface to call various LLMs (Ollama, Mistral, GPT-4) for agentic decisions.
- **LangChain & LangSmith:** The orchestrator and debugger.

LangChain manages the state and flow, while LangSmith provides full observability into the agent's "thought process."

The Ingestion Deep Dive

Before we can answer questions, we need to build the brain. The ingestion pipeline transforms raw text into a structured Knowledge Graph using the **GraphReader Extractor**.

1. Atomic Facts Extraction

Instead of just dumping text into the database, we use an LLM to distill "Atomic Facts" — the smallest indivisible units of information. This prevents the "lost in the middle" phenomenon common in large chunks.

Example: *Input Text:* “Keytruda is an immunotherapy that targets PD-1 to treat lung cancer.”

Extracted Atomic Facts (JSON):

```
{
  "atomic_facts": [
    {
      "key_elements": ["Keytruda", "Immunotherapy"],
      "atomic_fact": "Keytruda is a type of immunotherapy."
    },
    {
      "key_elements": ["Keytruda", "PD-1", "Lung Cancer"],
      "atomic_fact": "Keytruda targets PD-1 to treat lung cancer."
    }
  ]
}
```

2. Entity Resolution (Deduplication)

In real-world data, “Google”, “Google Inc.”, and “Alphabet” might refer to the same entity. We use **SpaCy** and **Fuzzy Search** to identify and merge these duplicates.

- **Technique:** Semantic Similarity & Levenshtein Distance.
- **Threshold:** We set a strict similarity threshold (e.g., 0.95) to avoid false positives.
- **Result:** A cleaner graph where all facts about “Google” are connected to a single node, not scattered across three.

3. Indexing Strategy

We don't just create one index; we create two for maximum recall:

1. **Vector Index:** Uses `cosine` similarity on 1024-dimensional embeddings (via Mistral/Ollama) to find conceptually similar facts.
2. **Keyword Index (Fulltext):** A standard Lucene index to catch exact matches (e.g., specific drug names or ID numbers) that vector search might miss.

```
Creating the Vector Index in Neo4j
CREATE VECTOR INDEX fact_embeddings IF NOT EXISTS
FOR (n:FactNode) ON (n.embedding)
OPTIONS {indexConfig: {
  `vector.dimensions`: 1024,
  `vector.similarity_function`: 'cosine'
}}
```

The Retrieval Deep Dive

1. The GraphReader Retriever

This is the agentic core. Unlike a standard retriever that just runs a query and returns results, the

GraphReaderRetriever is a class that implements the entire “Thinking Loop” (Discovery -> Traversal -> Evaluation).

2. Vector Search with Scoring

We don't just take the top results blindly. We analyze the **similarity score** to determine confidence.

```
# Inside graph_reader_retriever.py
def initial_discovery(self, state):
    # ... setup retriever ...
    results = retriever.search(query_vector=question_embedding, top_k=5)

    # Extract scores to gauge relevance
    scores = [item.score for item in results.items]
    print(f"Vector hits: {len(results.items)} (Scores: {scores})")

    if max(scores) < 0.7:
        print("Warning: Low confidence match. Expanding search...")
        # Trigger fallback or broader search
```

The Implementation

We use a modern, modular stack to build this “Cognitive Graph”. Each component was chosen for its specific strength in the pipeline.

- **Chonkie**: A specialized chunking library that breaks long documents into meaningful “chunks” (not just random splits), preserving semantic context.
- **Neo4j**: The brain. It stores both the text (Vector) and the connections (Graph) in a single database, allowing for hybrid retrieval.
- **LiteLLM**: The reasoning engine. It provides a unified interface to

call various LLMs (Ollama, Mistral, GPT-4) for agentic decisions.

- **LangChain:** The orchestrator. It manages the state and flow of our agentic graph.

The Ingestion Pipeline

Before we can answer questions, we need to build the brain. The ingestion pipeline transforms raw text into a structured Knowledge Graph.

The Workflow

The pipeline follows a strict sequence to ensure data quality:

1. **Chunking:** Documents are split into semantic blocks.
2. **Extraction:** An LLM identifies entities (e.g., “PD-1”) and atomic facts (e.g., “PD-1 inhibits T-cells”).
3. **Indexing:** Facts are embedded into vectors for fast search.
4. **Resolution:** We use SpaCy to merge duplicate entities (so “Google” and “Google Inc.” become one node).

Code Snippet: Building the Pipeline

Here is how we define this workflow using LangGraph:

```
def _build_workflow(self):  
    """Build LangGraph workflow for ingestion."""
```

```

workflow = StateGraph(GraphRAGState)

# Add all nodes
workflow.add_node("document_loader", self._load_documents)
workflow.add_node("text_chunker", self._chunk_documents)
workflow.add_node("entity_extractor", self._extract_graph)
workflow.add_node("fact_extractor", self._extract_atomic_facts)
workflow.add_node("fact_writer", self._write_atomic_facts)
workflow.add_node("vector_indexer", self._create_vector_index)
workflow.add_node("entity_resolver", self._resolve_entities)

# Define the flow
workflow.set_entry_point("document_loader")
workflow.add_edge("document_loader", "text_chunker")
workflow.add_edge("text_chunker", "entity_extractor")
workflow.add_edge("entity_extractor", "fact_extractor")
workflow.add_edge("fact_extractor", "fact_writer")
workflow.add_edge("fact_writer", "vector_indexer")
workflow.add_edge("vector_indexer", "entity_resolver")
workflow.add_edge("entity_resolver", END)

return workflow.compile()

```

The Implementation

Let us first look at the project scaffolding.

```

.
├── src
│   ├── GraphRAG
│   │   ├── Unstructured-kgpipeline-lexical
│   │   │   ├── langchain_graphrag_pipeline.py # Core Pipeline
│   │   │   ├── graph_reader_retriever.py      # The Agentic Retriever
│   │   │   ├── retrieval_chain.py             # Routing Logic
│   │   │   ├── langchain_config.yaml          # Configuration
│   │   └── requirements.txt
│   └── .env
├── data
│   └── medical_research.pdf

```

Stage 1: The Intelligent Router (retrieval_chain.py)

Here's where it gets interesting. Instead of treating every query the same, we use a lightweight LLM call to classify the question. This acts as a "Traffic Cop," directing simple queries to the Fast Path and complex ones to the Agent

```
def _classify_question_complexity(self, question: str) -> str:
    """Classify question complexity using LLM."""
    try:
        prompt = f"""Analyze this question and classify its complexity:
        Question: {question}
        Criteria:
        - SIMPLE: Single fact lookup, direct entity query (e.g., "What is...")
        - COMPLEX: Multi-hop reasoning, comparative analysis (e.g., "How...")
        Respond with ONLY a JSON object: {"complexity": "simple"} or {"complexity": "complex"}
        """
        response = self.llm.invoke(prompt, format="json")
        result = json.loads(response.content)
        return result.get("complexity", "simple").lower()
    except Exception:
        return "simple"
```

Stage 2: The Routing Logic (retrieval_chain.py)

Once classified, the system routes the query. Notice the robust fallback mechanism: if the Agentic path fails (e.g., due to a graph error), it automatically falls back to the standard Hybrid Retriever.

```
def answer_question(self, question: str, use_agentic: Optional[bool] = None):
    # 1. Determine Complexity
    if use_agentic is None:
        complexity = self._classify_question_complexity(question)
    else:
```

```

        complexity = "complex" if use_agentic else "simple"

# 2. Try GraphReader (Primary)
if self.agentic_enabled:
    try:
        return self.graph_reader_retriever.run_retrieval(question, c
    except Exception as e:
        print(f"[INFO] Falling back to standard hybrid retrieval")

# 3. Fallback (Secondary)
return self.hybrid_retriever.answer_question(question)

```

Stage 3: The Agentic Core (graph_reader_retriever.py)

This is the brain of the operation. The

GraphReader isn't just a function; it's a state machine. It has nodes for **Discovery**, **Analysis**, and **Evaluation**.

The Hop Analyzer (The Explorer) This node performs the “thinking.” It looks at the facts it has found and decides where to go next in the graph.

```

def hop_analyzer(self, state: RetrievalState) -> RetrievalState:
    """Agentic Traversal: Performs undirected hop to find related facts.
    # Cypher query to find neighbors
    query = """
        MATCH (f:FactNode)-[:HAS_ENTITY]->(e:EntityNode)<-[:HAS_ENTITY]-
        WHERE f.fact IN $current_facts
        RETURN DISTINCT neighbor.fact AS new_fact
    """
    # ... execute query and update state ...
    return state

```

The Evaluator (The Judge) Before answering, the agent pauses to check its work. Does it have enough information? If not, it loops back.

```
def evaluate_answer(self, state: RetrievalState) -> dict:
    """Decides if answer is sufficient, needs deepdive, or needs more hops
    prompt = f"""
    Original Question: {state['original_question']}
    Current Facts: {json.dumps(state['notebook'])}

    Is the question answered?
    1. 'sufficient': Answer is complete.
    2. 'hop_more': Need more related facts.
    3. 'deepdive': Need a new line of questioning.
    """
    # ... invoke LLM and return decision ...
```

The Result: Multi-Hop Reasoning in Action

Let's look at a real execution trace for a complex medical query:

“How does immunotherapy target PD-1 to treat lung cancer?”

Step 1: Initial Discovery

The system finds the entry point nodes.

- *Found*: “PD-1 is an immune checkpoint receptor.”
- *Found*: “Immunotherapy uses checkpoint inhibitors.”

Step 2: The Hop (Graph Traversal)

The **Hop Analyzer** sees a connection between “PD-1” and “T-cells”. It decides to follow this path.

- *Hop*: “PD-1 acts as a brake on T-cells.”
- *Hop*: “Blocking PD-1 releases this brake.”

Step 3: The Synthesis (Final Answer)

The **Context Manager** compiles these facts into the final response.

Final Answer: Immunotherapy targets PD-1, a checkpoint protein on immune T-cells, to treat lung cancer by blocking its interaction with PD-L1 on cancer cells. Normally, this interaction acts as an “off switch” that prevents T-cells from attacking the cancer. By using **checkpoint inhibitors** (a form of immunotherapy), this signal is blocked, allowing the T-cells to recognize and destroy the lung cancer cells.

The Conclusion

This hands-on implementation demonstrates how a GraphRAG pipeline fundamentally elevates the quality and reliability of enterprise AI systems. By combining **Chonkie’s** semantic chunking, **Neo4j’s** graph capabilities, and **Agentic Routing**, we built a stack that can see, read, and reason with far greater precision than text-only systems.

The result is a robust workflow where “thinking” is a first-class

citizen. The intelligent routing strategy ensures that every query gets exactly the amount of compute it needs — fast answers for simple questions, and deep reasoning for complex ones. This architecture proves that the next evolution of RAG is not just about better embeddings — it's about **cognitive architecture**.

Neo4j

Neo4j Graph Database

Agentic Ai

Chunking

Langchain



Written by Akash Goyal

Follow

544 followers · 211 following

Building Agentic AI applications | Generative AI | LLMs | Agents | NLP | Data Scientist | Machine Learning <https://www.linkedin.com/in/akash-g-7a5224246/>

Responses (2)



To respond to this story,
get the free Medium app.



Kevin Hughes

6 days ago



Which Akash Goyal are you on linked in or elsewhere?



1



Jack Park

Dec 22



I get the impression that you are homing in on an architecture worthy of committing to code in a repo.



More from Akash Goyal

 Akash Goyal

Building Agentic GraphOS: The 16-Layer Architecture Behind...

A Cost-Optimized, Fully Observable GraphRAG Operating System Built for...

★ Dec 14 🖱 507 💬 6 📌 ⋮

 Akash Goyal

Why LLMs Keep Forgetting Ontologies (And How to Fix It)

Production Architecture for Ontology Memory (Neo4j + LangGraph)

★ Dec 23 🖱 232 💬 3 📌 ⋮

 Akash Goyal

The Jaguar Problem: The Ontology Nightmare Haunting...

“Is my knowledge graph actually preserving information?” A deep dive...

★ Dec 11 🖱 260 💬 1 📌 ⋮

 Akash Goyal

A Self-Optimizing GraphRAG That Selects the Right Retrieval...

“How cognitive routing, graph reasoning, and cost-aware decisioning outperform...

★ Dec 13 🖱 238 💬 1 📌 ⋮

See all from Akash Goyal

Recommended from Medium



Pankaj Kumar

Building Your First Ontology: A Hands-On Tutorial

Stop Reading About Ontology. Start Building One.



Dec 13



243



Akash Goyal

Build time-aware knowledge graphs with Temporal Agents

How temporal reasoning and dual-track extraction create knowledge graphs tha...



Dec 20



183



3



 In AI Exploration Journey by Florian June

Building Smarter Knowledge Graphs with Tree-KG

Most knowledge graphs feel like spreadsheets with edges, what would...

★ Dec 22 🖱 29 💬 1 📌 ⋮

 In GoPenAI by Greek Ai

🚀 NVIDIA Nemotron 3 Nano: The Small Model Quietly...

For years, NVIDIA was the company behind AI—but only behind the scenes....

★ Dec 23 🖱 241 💬 1 📌 ⋮

 In Towards AI by Divy Yadav

9 RAG Architectures Every AI Developer Must Know: A...

Architectures beyond Naive Rag to build reliable production AI Systems

Dec 19 🖱 351 💬 8 📌 ⋮

 In Level Up Coding by Fareed Khan

Building the 7 Layers of a Production-Grade Agentic AI...

Service Layer, Middleware, Context Management and more

★ Dec 19 🖱 1.2K 💬 17 📌 ⋮

See more recommendations

